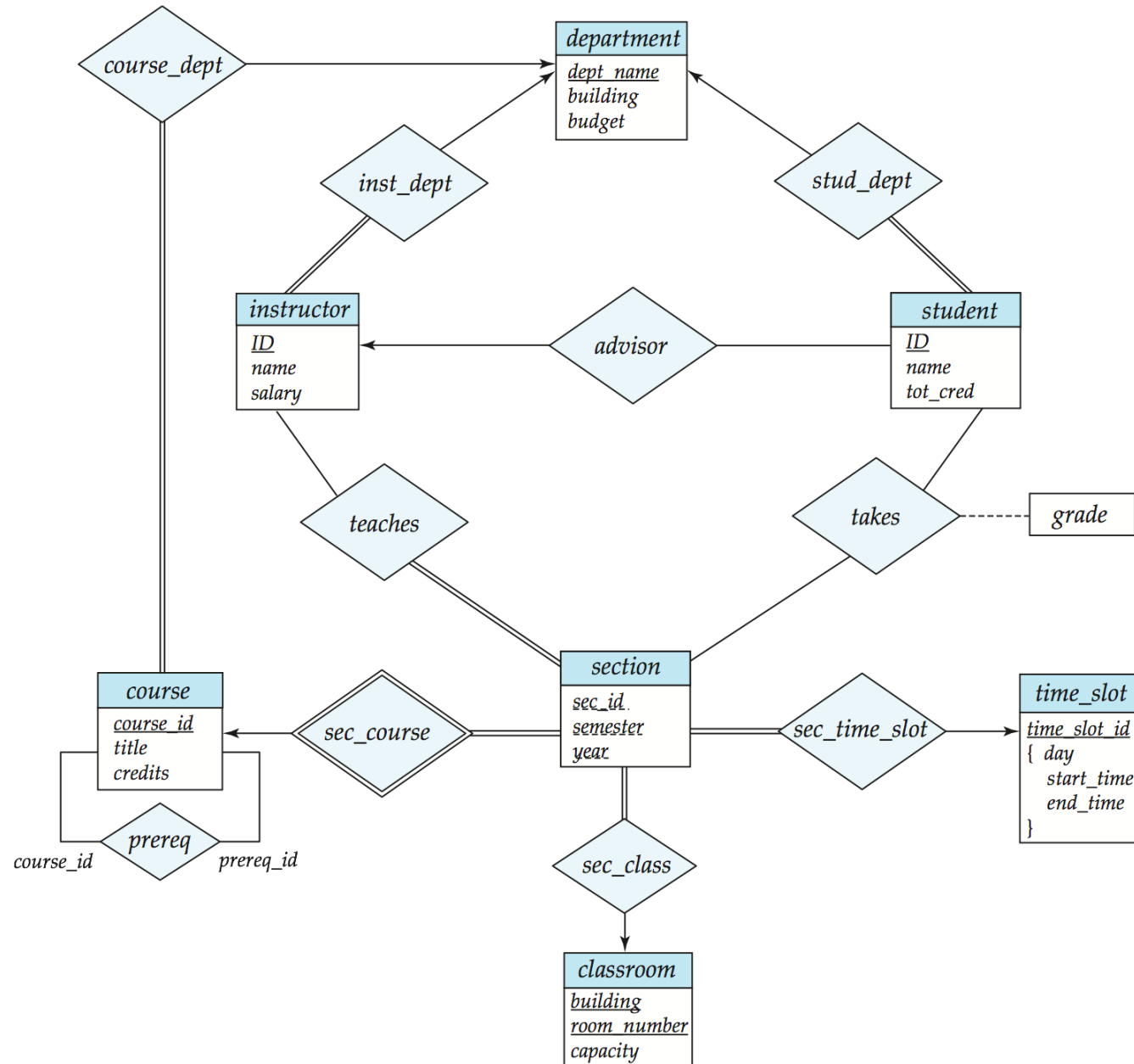


Chapter 7: Relational Database Design

Outline

- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Decomposition Using Functional Dependencies
- Functional Dependency Theory
- Algorithms for Functional Dependencies
- Decomposition Using Multivalued Dependencies
- More Normal Form
- Database-Design Process
- Modeling Temporal Data

E-R Diagram for a University Enterprise



Reduce to Relational Schemas

- Relational Schemas of a University Enterprise

classroom(building, room_number, capacity)

department(dept_name, building, budget)

course(course_id, title, dept_name, credits)

instructor(ID, name, dept_name, salary)

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

teaches(ID, course_id, sec_id, semester, year)

student(ID, name, dept_name, tot_cred)

takes(ID, course_id, sec_id, semester, year, grade)

advisor(s_ID, i_ID)

time_slot(time_slot_id, day, start_time, end_time)

prereq(course_id, prereq_id)

- What about **combining** instructor and department ?
- What about **combining** instructor and teaches?
- What about **splitting** student → S1(ID, department) , S2(name, tot_cred)?

Combine Schemas?

- Suppose we combine **instructor** and **department** into **inst_dept**
- Result is possible repetition of information

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- Pitfalls of the “bad” relations
 - **Information repetition** (信息重复)
 - **Insertion anomalies** (插入异常)
 - **Update difficulty** (更新困难)

What About Smaller Schemas?

- Suppose we had started with

inst_dept(id, name, salary, dept_name, building, budget)

- How would we know to split up (decompose) it into instructor and department?

if there were a schema **department** (*dept_name*, *building*, *budget*), then ***dept_name*** would be a *candidate key*”

- Denote as a functional dependency:

id → *name*, *salary*, *dept_name*

dept_name → *building*, *budget*

- In *inst_dept*, because ***dept_name*** is not a candidate key, the ***building*** and ***budget*** of a **department** may have to be repeated.

This indicates the need to decompose *inst_dept*

What About Smaller Schemas?

- Not all decompositions are good. Suppose we decompose

employee(*ID*, *name*, *street*, *city*, *salary*)

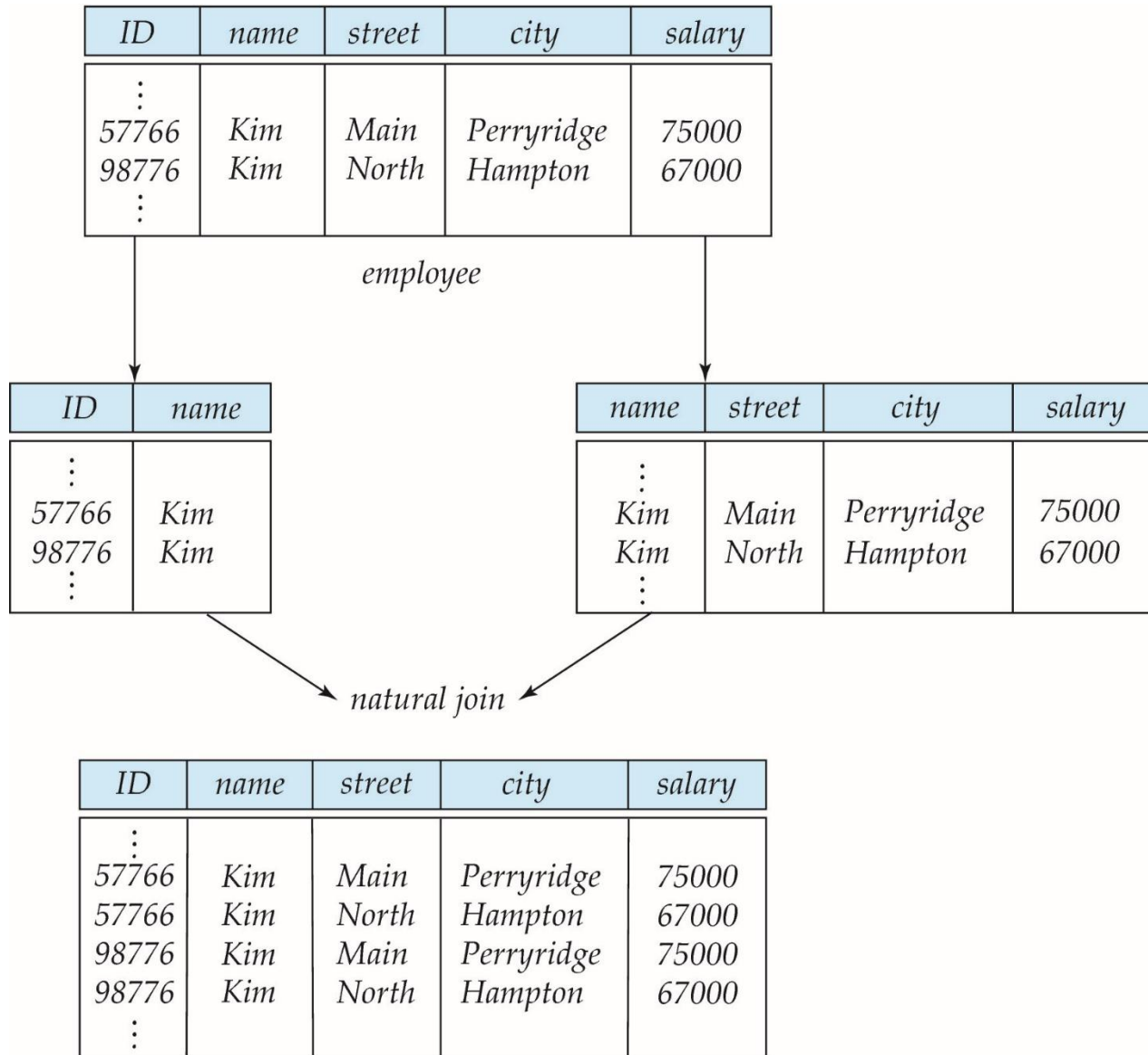
→

employee1 (*ID*, *name*)

employee2 (*name*, *street*, *city*, *salary*)

The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**.

A Lossy Decomposition



Example of Lossless-Join Decomposition

- Lossless join decomposition

Decomposition of $R = (\underline{A}, B, C)$

$$R_1 = (\underline{A}, B), R_2 = (\underline{B}, C)$$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B

Lossless-join Decomposition

Let R be a relation schema and let R_1 and R_2 form a **decomposition** of R .
That is $R = R_1 \cup R_2$

We say that the decomposition is a **lossless decomposition** if there is no loss of information by replacing R with the two relation schemas $R_1 \cup R_2$.
Formally, $\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$

And, conversely a decomposition is **lossy** if

$$r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

Note: more tuples implies more uncertainty (less information).

- A **decomposition** of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies holds:

$$R_1 \cap R_2 \rightarrow R_1$$

$$R_1 \cap R_2 \rightarrow R_2$$

Goal — Devise a Theory for the Following

- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into a set of relations {R1, R2, ..., Rn} such that
 - each relation is in **good form**
 - the **decomposition** is a **lossless-join** decomposition
- Our theory is based on:
 - **functional dependencies**
 - **multivalued dependencies**
- **Normal Forms(NF):**
 - 1NF → 2NF → 3NF → **BCNF** → 4NF

Functional Dependencies

Functional Dependencies

- There are usually a variety of **constraints** (rules) on the data in the real world.
- For example, some of the **constraints** that are expected to hold in a university database are:

Students are **uniquely identified** by their ID.

Each student has **only one** name.

Each student is (primarily) **associated with only one** department.

Each department has **only one** value for its budget, and **only one** associated building.

- Above **constraints** are captured in the **ER diagram** of the university database.

An instance of a relation that satisfies all such real-world constraints is called **a legal instance** of the relation;

A **legal instance of a database** is one where all the relation instances are legal instances

Functional Dependencies (Cont.)

Functional Dependencies are **constraints** on the set of legal relations.

Require that the value for a certain set of attributes **determines uniquely** the value for another set of attributes.

dept_name* → *building

A functional dependency is a generalization of the notion of a **key**.

Functional Dependencies (Cont.)

- Let R be a relation schema

$$\alpha \subseteq R \text{ and } \beta \subseteq R$$

The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β . That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A,B)$ with the following instance of r .

1	4
1	5
3	7

On this instance, $A \rightarrow B$ does **NOT** hold, but $B \rightarrow A$ does hold.

Functional Dependencies (Cont.)

- K is a superkey for relation schema R **if and only if** $K \rightarrow R$
- K is a candidate key for R if and only if
 - $K \rightarrow R$, and
 - for no** $\alpha \subset K$, $\alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

inst_dept (ID, name, salary, dept_name, building, budget).

We expect these functional dependencies to hold:

dept_name* → *building

ID* → *building

but would not expect the following to hold:

dept_name* → *salary

Functional Dependencies (Cont.)

- A functional dependency is **trivial** if it is satisfied by all relations

Example:

- ▶ $ID, name \rightarrow ID$
- ▶ $name \rightarrow name$

In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

Closure(闭包) of a Set of Functional Dependencies

- Given a set F of functional dependencies, there are certain other functional dependencies that are logically implied by F .

For example: If $A \rightarrow B$ and $B \rightarrow C$,

then we can infer that $A \rightarrow C$

The set of **all** functional dependencies **logically implied** by F is the **closure** of F .

- We denote the closure of F by F^+ .

F^+ is a superset of F .

$F = \{A \rightarrow B, B \rightarrow C\}$, what is F^+ ?

$F^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C, AB \rightarrow B, AB \rightarrow C, \dots\}$

Closure of a Set of Functional Dependencies

- We can find F^+ , the closure of F , by repeatedly applying **Armstrong's Axioms**:
 - if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (**reflexivity**, 自反率)
 - if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$ (**augmentation**, 增补率)
 - if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (**transitivity**, 传递率)
- These rules are
 - Sound** (正确有效的) (generate only functional dependencies that actually hold), and
 - Complete** (完备的) (generate all functional dependencies that hold).

Example

$R = (A, B, C, G, H, I)$

$F = \{$
 $A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

some members of F^+

$A \rightarrow H$

- ▶ by transitivity from $A \rightarrow B$ and $B \rightarrow H$

$AG \rightarrow I$

- ▶ by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$
and then transitivity with $CG \rightarrow I$

$CG \rightarrow HI$

- ▶ by augmenting $CG \rightarrow I$ to infer $CG \rightarrow CGI$,
and augmenting of $CG \rightarrow H$ to infer $CGI \rightarrow HI$,
and then transitivity

Closure of Functional Dependencies (Cont.)

- Additional rules:

If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds

(**union**, 合并)

If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds

(**decomposition**, 分解)

If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds

(**pseudotransitivity**, 伪传递)

- The above rules can be inferred from Armstrong's axioms.

Exercise 1

- The functional dependency $\alpha \gamma \rightarrow \beta \gamma$ holds on $\mathbf{R}(\alpha, \beta, \gamma)$, prove that $\alpha \gamma \rightarrow \beta \gamma$ is equivalent with $\alpha \gamma \rightarrow \beta$.

Duplicated attributes can be removed from right side of a functional dependency.

Prove:

$$\alpha \gamma \rightarrow \beta \gamma, \quad \beta \gamma \rightarrow \beta \quad \implies \alpha \gamma \rightarrow \beta$$

(reflexivity)

$$\alpha \gamma \rightarrow \beta \implies \alpha \gamma \gamma \rightarrow \beta \gamma \implies \alpha \gamma \rightarrow \beta \gamma$$

(Augmentation)

Closure of Attribute Sets

- Given a set of attributes $\mathbf{a}$, define the closure of $\mathbf{a}$ under F (denoted by $\mathbf{a}^+$) as the set of attributes that are functionally determined by $\mathbf{a}$ under F

$R(A,B,C,D) \quad F=\{A \rightarrow B, B \rightarrow C, B \rightarrow D\}$

$A^+=ABCD$

$B^+=BCD$

$C^+=C$

Algorithm to compute α^+ , the closure of α under F

```
result :=  $\alpha$ ;  
while (changes to result) do  
  for each  $\beta \rightarrow \gamma$  in  $F$  do  
    begin  
      if  $\beta \subseteq \textit{result}$  then  $\textit{result} := \textit{result} \cup \gamma$   
    end
```

Example of Attribute Set Closure

$R = (A, B, C, G, H, I)$

$F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

$(AG)^+$

1. $result = AG$
2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)

Is AG a candidate key?

1. Is AG a super key?

1. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$

2. Is any subset of AG a superkey?

1. Does $A \rightarrow R$? == Is $(A)^+ \supseteq R$

2. Does $G \rightarrow R$? == Is $(G)^+ \supseteq R$

Uses of Attribute Closure

- There are several uses of the attribute closure algorithm:

- **Testing for superkey:**

To test if α is a superkey, we compute α^+ and check if α^+ contains all attributes of R .

- **Testing functional dependencies**

To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.

That is, we compute α^+ by using attribute closure, and then check if it contains β .

Is a simple and cheap test, and very useful

- **Computing closure of F**

For each $\gamma \subseteq R$, we find the closure γ^+ , and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.

Uses of Attribute Closure

- Computing closure of F:

$R(\underline{A}, B, C)$, $F = \{A \rightarrow B, B \rightarrow C\}$

computing:

$A^+ = ABC$, $B^+ = BC$, $C^+ = C$

$(AB)^+ = ABC$, $(AC)^+ = ABC$, $(BC)^+ = BC$, $(ABC)^+ = ABC$

$F \Leftrightarrow \{A \rightarrow ABC, B \rightarrow BC, C \rightarrow C, AB \rightarrow ABC, AC \rightarrow ABC, BC \rightarrow BC, ABC \rightarrow ABC\}$

$\Leftrightarrow \{A \rightarrow A, A \rightarrow B, A \rightarrow C, A \rightarrow AB, A \rightarrow AC, A \rightarrow BC, A \rightarrow ABC$

$B \rightarrow B, B \rightarrow C, B \rightarrow BC,$

$C \rightarrow C,$

$AB \rightarrow A, AB \rightarrow B, AB \rightarrow C, AB \rightarrow AB, AB \rightarrow AC, AB \rightarrow BC, AB \rightarrow ABC,$

$AC \rightarrow A, AC \rightarrow B, AC \rightarrow C, AC \rightarrow AB, AC \rightarrow AC, AC \rightarrow BC, AC \rightarrow ABC,$

$BC \rightarrow B, BC \rightarrow C, BC \rightarrow BC,$

$ABC \rightarrow A, ABC \rightarrow B, ABC \rightarrow C, ABC \rightarrow AB, ABC \rightarrow AC, ABC \rightarrow BC, ABC \rightarrow ABC\}$

$= F^+$

Canonical Cover (正则覆盖)

- Sets of functional dependencies may have redundant dependencies that can be inferred from the others

For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

- Parts of a functional dependency may be redundant

E.g.: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$

can be simplified to

$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

$A \rightarrow CD \Rightarrow A \rightarrow D$

$A \rightarrow B, B \rightarrow C \Rightarrow A \rightarrow C, A \rightarrow D \Rightarrow A \rightarrow CD$

E.g.: on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to
 $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

$A \rightarrow B, B \rightarrow C \Rightarrow A \rightarrow C \Rightarrow A \rightarrow AC, AC \rightarrow D \Rightarrow A \rightarrow D$

$A \rightarrow D \Rightarrow AC \rightarrow CD \Rightarrow AC \rightarrow D$

- Intuitively, a canonical cover of F is a “minimal” set of functional dependencies equivalent to F , having no redundant dependencies or redundant parts of dependencies

Extraneous Attributes(无关属性)

- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .

Attribute A is **extraneous** in α if $A \in \alpha$
and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.

Attribute A is **extraneous** in β if $A \in \beta$
and the set of functional dependencies
 $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F .

Note: implication in the opposite direction is trivial in each of the cases above, since a “stronger” functional dependency always implies a weaker one

Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$

B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$ (i.e. the result of dropping B from $AB \rightarrow C$).

Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$

C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C

Canonical Cover

- A canonical cover for F is a set of dependencies F_c such that
 - F logically implies all dependencies in F_c , and
 - F_c logically implies all dependencies in F , and
 - No functional dependency in F_c contains an extraneous attribute, and
 - Each left side of functional dependency in F_c is unique.
- To compute a canonical cover for F :
 - repeat**
 - Use the union rule to replace any dependencies in F
 $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$
 - Find a functional dependency $\alpha \rightarrow \beta$ with an
extraneous attribute either in α or in β
 - If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$
 - until** F does not change
- Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied

Computing a Canonical Cover

- $R = (A, B, C)$
 $F = \{A \rightarrow BC$
 $B \rightarrow C$
 $A \rightarrow B$
 $AB \rightarrow C\}$
- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 Set is now $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- A is extraneous in $AB \rightarrow C$
 Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies
 - ▶ Yes: in fact, $B \rightarrow C$ is already present! Set is now $\{A \rightarrow BC, B \rightarrow C\}$
- C is extraneous in $A \rightarrow BC$
 Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies
 - ▶ Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
 - Can use attribute closure of A in more complex cases
- The canonical cover is: $\{ A \rightarrow B$
 $B \rightarrow C \}$

Computing a Canonical Cover

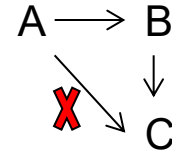
$R = (A, B, C)$

$F = \{A \rightarrow BC$

$B \rightarrow C$

$A \rightarrow B$

$AB \rightarrow C\}$



The canonical cover is: $\{A \rightarrow B, B \rightarrow C\}$

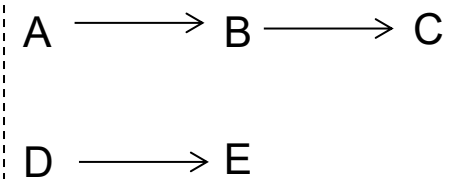
Exercise 2

For relation schema $R(A,B,C,D,E)$ with functional dependencies set $F=\{A \rightarrow BC, AD \rightarrow E, B \rightarrow C, D \rightarrow E\}$.

- a) Compute canonical cover F_c .
- b) Compute $(AE)^+$
- c) Find all candidate keys of R

Answers

- a) $F_c=\{A \rightarrow B, B \rightarrow C, D \rightarrow E\}$.
- b) $(AE)^+ = ABCE$
- c) Candidate key: AD



Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of functional dependencies if for all functional dependencies in F^+ of the form

$$\alpha \rightarrow \beta$$

where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$)
 - α is a superkey for R
- Example schema not in BCNF:

instr_dept (ID, name, salary, dept_name, building, budget)

because ***dept_name* $\rightarrow$ building, budget**
holds on *instr_dept*, but *dept_name* is not a superkey

Decomposing a Schema into BCNF

- Suppose we have a schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF.

We decompose R into:

- $(\alpha \cup \beta)$
- $(R - (\beta - \alpha))$

- In our example,

$\alpha = \text{dept_name}$

$\beta = \text{building, budget}$

and *inst_dept* is replaced by

$(\alpha \cup \beta) = (\text{dept_name, building, budget})$

$(R - (\beta - \alpha)) = (ID, name, salary, dept_name)$

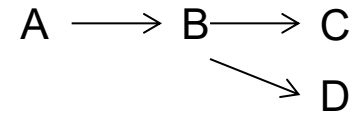
BCNF Decomposition Algorithm

```
1  result := {R };
2  done := false;
3  while (not done) do
4      if (there is a schema  $R_i$  in result that is not in BCNF)
5          then begin
6              let  $\alpha \rightarrow \beta$  be a nontrivial functional dependency that
7              holds on  $R_i$  such that  $\alpha^+$  does not contain  $R_i$  and  $\alpha \cap \beta = \emptyset$ ;
8              result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );
9          end
10     else done := true;
```

Note: each R_i is in BCNF, and decomposition is lossless-join.

Exercise 3

- For the relation schema $R(A,B,C,D)$ with the functional dependencies set $F=\{A \rightarrow B, B \rightarrow CD\}$,
 - a) List all candidate keys of the relation.
 - b) Decompose the relation into a collection of BCNF relations. The decomposition must be lossless-join.



- Answer:

- a) candidate keys: A
- b) $R_1=(\underline{B},C,D), R_2=(\underline{A},B)$.

$F_1=\{B \rightarrow CD\}, F_2=\{A \rightarrow B\}$

The decomposition is lossless-join ,

because $R_1 \cap R_2 = (B)$, and B is a candidate key of R_1 .

BCNF and Dependency Preservation

- Constraints, including **functional dependencies**, are **costly to check** in practice **unless they pertain to only one relation**

If it is sufficient to test only those dependencies on each individual relation of a decomposition in order to ensure that *all* functional dependencies hold, then that decomposition is **dependency preserving** (保持依赖).

(如果通过检验单一关系上的函数依赖, 就能确保所有的函数依赖成立, 那么这样的分解是依赖保持的)

(或者, 原来关系 R 上的每一个函数依赖, 都可以在分解后的单一关系上得到检验或者推导得到。)

- Because it is not always possible to achieve both BCNF and dependency preservation, we consider a weaker normal form, known as third normal form.

Dependency Preservation

Let F_i be the set of all functional dependencies in F^+ that include only attributes in R_i . (F_i : *the restriction of F on R_i*)

- ▶ A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

- ▶ If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive.

Example

$R = (A, B, C)$

$F = \{A \rightarrow B$

$B \rightarrow C\}$

$A \longrightarrow B \longrightarrow C$

Key = $\{A\}$

R is not in BCNF

Decomposition $R_1 = (A, B), R_2 = (B, C)$

$F_1 = \{A \rightarrow B\}, F_2 = \{B \rightarrow C\}$

R_1 and R_2 in BCNF

Lossless-join decomposition

Dependency preserving, since $(F_1 \cup F_2)^+ = F^+$

Decomposition $R_1 = (A, B), R_2 = (A, C)$

$F_1 = \{A \rightarrow B\}, F_2 = \{A \rightarrow C\}$

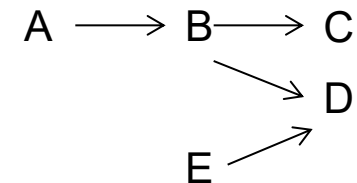
R_1 and R_2 in BCNF

Lossless-join decomposition

Not dependency preserving, since $(F_1 \cup F_2)^+ \neq F^+$

Exercise 4

- For the relation schema $R(A,B,C,D,E)$ with the functional dependencies set $F=\{A \rightarrow B, B \rightarrow CD, E \rightarrow D\}$,
 - a) List all **candidate keys** of the relation.
 - b) Decompose the relation into a collection of **BCNF** relations. The decomposition must be **lossless-join**.
 - c) Whether the decomposition of (b) is **dependency preserving** or not?



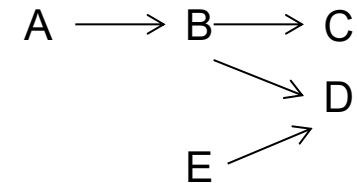
- **Answer One:**

- a) candidate keys: AE
- b) $R1(\underline{E}, D)$, $R2(\underline{B}, C)$, $R3(\underline{A}, B)$, $R4(\underline{A}, \underline{E})$
- c) Above decomposition is not dependency preserving, because $\mathbf{B \rightarrow D}$ cannot be inferred by all functional dependencies holds on $R1$, $R2$, $R3$, and $R4$.

$$F1=\{E \rightarrow D\} \quad F2=\{B \rightarrow C\} \quad F3=\{A \rightarrow B\} \quad F4=\{AE \rightarrow AE\}$$

Exercise 4

- For the relation schema $R(A,B,C,D,E)$ with the functional dependencies set $F=\{A \rightarrow B, B \rightarrow CD, E \rightarrow D\}$,
 - a) List all candidate keys of the relation.
 - b) Decompose the relation into a collection of BCNF relations. The decomposition must be lossless-join.
 - c) Whether the decomposition of (b) is dependency preserving or not?



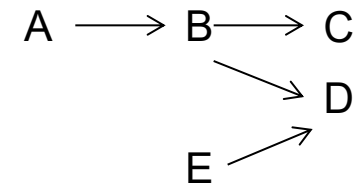
- Answer Two:

- a) candidate keys: AE
- b) $R1(\underline{B}, C, D)$, $R2(\underline{A}, B)$, $R3(\underline{A}, \underline{E})$
- c) Above decomposition is not dependency preserving, because $E \rightarrow D$ cannot be inferred by all functional dependencies holds on $R1, R2, R3$.

$$F1=(B \rightarrow CD) \quad F2=\{A \rightarrow B\} \quad F3=\{AE \rightarrow AE\}$$

Exercise 4

- For the relation schema $R(A,B,C,D,E)$ with the functional dependencies set $F=\{A \rightarrow B, B \rightarrow CD, E \rightarrow D\}$,
 - a) List all candidate keys of the relation.
 - b) Decompose the relation into a collection of BCNF relations. The decomposition must be lossless-join.
 - c) Whether the decomposition of (b) is dependency preserving or not?



- Answer Three:

- a) candidate keys: AE
- b) $R1(\underline{B}, C, D)$, $R2(\underline{A}, B)$, $R3(\underline{E}, D)$, $R4(\underline{A}, \underline{E})$

The **decomposition is lossless-join**, because

$$((R1(\underline{B}, C, D) \bowtie R2(\underline{A}, B)) \bowtie R4(\underline{A}, \underline{E})) \bowtie R3(\underline{E}, D) = R(A, B, C, D, E)$$

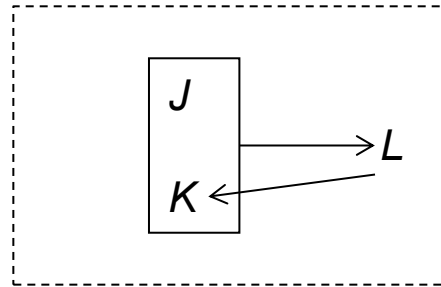
(note : the decomposition can not be got from the BCNF Decomposition Algorithm.)

- a) Above decomposition is **dependency preserving**, because each functional dependency of F can be checked in single relation of $R1$, $R2$, $R3$ or $R4$.

BCNF and Dependency Preservation

It is not always possible to get a BCNF decomposition that is dependency preserving

- $R = (J, K, L)$
 $F = \{JK \rightarrow L$
 $L \rightarrow K\}$



- Two candidate keys = JK and JL
- R is not in BCNF
- Decomposition R into $R_1(L, K)$, $R_2(J, L)$
- All decomposition of R will fail to preserve

$$JK \rightarrow L$$

This implies that testing for $JK \rightarrow L$ requires a join

Third Normal Form: Motivation

There are some situations where

BCNF is not dependency preserving, and
efficient checking for FD violation on updates is important

Solution: define a weaker normal form, called Third Normal Form (3NF)

Allows some redundancy

But functional dependencies can be checked on individual relations without computing a join.

There is always a lossless-join, dependency-preserving decomposition into 3NF.

Third Normal Form

- A relation schema R is in third normal form (3NF) if for all:

$$\alpha \rightarrow \beta \text{ in } F^+$$

at least one of the following holds:

- $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \in \alpha$)
- α is a superkey for R
- **Each attribute A in $\beta - \alpha$ is contained in a candidate key for R .**

(**NOTE:** each attribute may be in a different candidate key)

- If a relation is in BCNF it is in 3NF (since in BCNF one of the first two conditions above must hold).
- Third condition is a minimal relaxation of BCNF to ensure dependency preservation.

3NF Example

Relation *dept_advisor*:

dept_advisor (s_ID, i_ID, dept_name)

$F = \{s_ID, dept_name \rightarrow i_ID, i_ID \rightarrow dept_name\}$

Two candidate keys: s_ID, dept_name, and i_ID, s_ID

R is in 3NF

- ▶ s_ID, dept_name $\rightarrow$ i_ID
 - s_ID, dept_name is a superkey
- ▶ i_ID $\rightarrow$ dept_name
 - dept_name is contained in a candidate key

3NF Decomposition Algorithm

```
1  Let  $F_c$  be a canonical cover for  $F$ ;  
2   $i := 0$ ;  
3  for each functional dependency  $\alpha \rightarrow \beta$  in  $F_c$  do  
4      begin  
5           $i := i + 1$ ;     $R_i := \alpha \beta$   
6      end  
  
7  if none of the schemas  $R_j$ ,  $1 \leq j \leq i$  contains a candidate key for  $R$   
8  then begin  
9       $i := i + 1$ ;  
10      $R_i :=$  any candidate key for  $R$ ;  
11 end  
  
12 /* Optionally, remove redundant relations */  
13 repeat  
14     if any schema  $R_j$  is contained in another schema  $R_k$   
15     then /* delete  $R_j$  */  
16         begin  
17              $R_j = R_i$ ;   $i=i-1$ ;  
18 until no more  $R_j$  s can be deleted  
19 return ( $R_1, R_2, \dots, R_i$ )
```

Comparison of BCNF and 3NF

It is always possible to **decompose** a relation into a set of relations that are in **3NF** such that:

- the decomposition is lossless

- the dependencies are preserved

It is always possible to **decompose** a relation into a set of relations that are in **BCNF** such that:

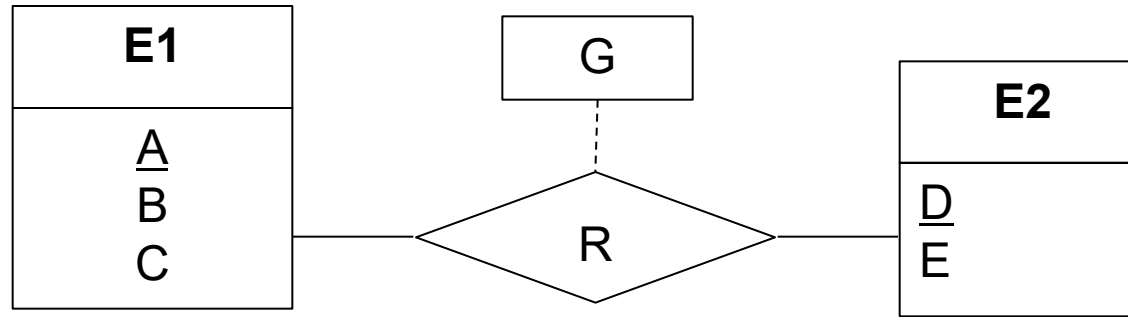
- the decomposition is lossless

- it may not be possible to preserve dependencies.

Goals of Normalization

- Let R be a relation scheme with a set F of functional dependencies. Decide whether a relation scheme R is in “good” form.
- In the case that a relation scheme R is not in “good” form, decompose it into a set of relation scheme $\{R_1, R_2, \dots, R_n\}$ such that
 - each relation scheme is in **good form (i.e., BCNF or 3NF)**
 - the decomposition is a **lossless-join decomposition**
 - Preferably, the decomposition should be **dependency preserving**.

E-R Modeling and Normal Forms

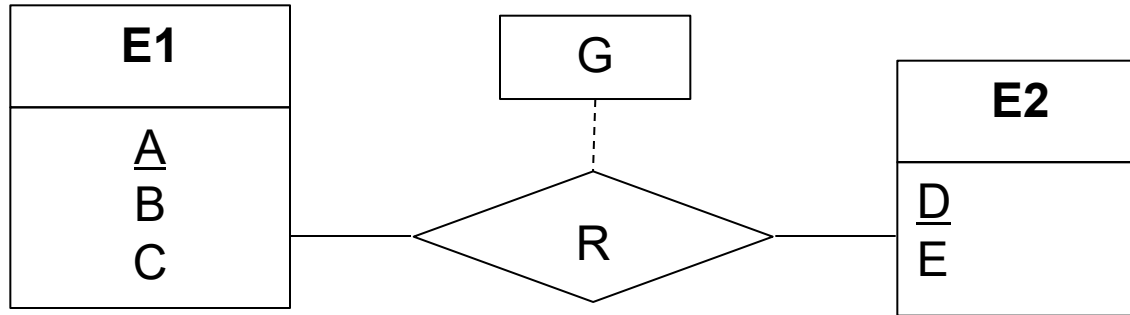


- Above is an E-R diagram that depicts a many-to-many relationship R between two entity sets E1 and E2. The candidate key of E1 and E2 are underlined in the diagram. If the E-R diagram is transform to a relation schema $S(A, B, C, D, E, G)$.

Please answer following questions:

1. Give the no-trivial functional dependency set F that holds on S.
2. List the candidate key(s) of S.
3. Prove that S is not in BCNF.
4. Explain the issues exist with S.
5. Prove that decomposing of S into three relation schemas $R1(A,B,C)$, $R2(D,E)$ and $R3(A,D,G)$ is lossless join.

E-R Modeling and Normal Forms



■ Please answer following questions:

1. Give the no-trivial functional dependency set F that holds on S.

$F = \{A \rightarrow BC, D \rightarrow E, AD \rightarrow G\}$

2. List the candidate key(s) of S.

ADG

3. Prove that S is not in BCNF.

$A \rightarrow BC$, A is not a key.

4. Explain the issues exist with S.

Information repetition, Insertion anomalies, Update difficulty

5. Prove that decomposing of S into three relation schemas

$R = R_1(A, B, C) \bowtie (R_2(D, E) \bowtie R_3(A, D, G))$

How good is BCNF?

- There are database schemas in BCNF that do not seem to be sufficiently normalized
- Consider a relation

inst_info (ID, child_name, phone)

where an instructor may have more than one phone and can have multiple children

<i>ID</i>	<i>child_name</i>	<i>phone</i>
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	Willian	512-555-4321

inst_info

How good is BCNF? (Cont.)

- There are no non-trivial functional dependencies and therefore the relation is in BCNF

Insertion anomalies – i.e., if we add a phone 981-992-3443 to 99999, we need to add two tuples

(99999, David, 981-992-3443)
(99999, William, 981-992-3443)

How good is BCNF? (Cont.)

- Therefore, it is better to decompose inst_info into:

<i>inst_child</i>	<i>ID</i>	<i>child_name</i>
	99999	David
	99999	Willian

<i>inst_phone</i>	<i>ID</i>	<i>phone</i>
	99999	512-555-1234
	99999	512-555-4321

This suggests the need for higher normal forms, such as **Fourth Normal Form (4NF)**.

Multivalued Dependencies

Multivalued Dependencies

- Suppose we record names of children, and phone numbers for instructors:

inst_child(ID, child_name)

inst_phone(ID, phone_number)

- If we were to combine these schemas to get

inst_info(ID, child_name, phone_number)

Example data:

(99999, David, 512-555-1234)

(99999, David, 512-555-4321)

(99999, William, 512-555-1234)

(99999, William, 512-555-4321)

- This relation is in BCNF

Why?

Multivalued Dependencies (MVDs)

- Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency**

$$\alpha \twoheadrightarrow \beta$$

holds on R if in any legal relation $r(R)$, for all pairs for tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:

$$\begin{aligned} t_3[\alpha] &= t_4[\alpha] = t_1[\alpha] = t_2[\alpha] \\ t_3[\beta] &= t_1[\beta] \\ t_3[R - \beta] &= t_2[R - \beta] \\ t_4[\beta] &= t_2[\beta] \\ t_4[R - \beta] &= t_1[R - \beta] \end{aligned}$$

MVD (Cont.)

- Tabular representation of $\alpha \twoheadrightarrow \beta$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Example

- Let R be a relation schema with a set of attributes that are partitioned into 3 nonempty subsets.

Y, Z, W

- We say that $Y \twoheadrightarrow Z$ (Y multidetermines Z) if and only if for all possible relations $r(R)$

$\langle y_1, z_1, w_1 \rangle \in r$ and $\langle y_1, z_2, w_2 \rangle \in r$

then

$\langle y_1, z_1, w_2 \rangle \in r$ and $\langle y_1, z_2, w_1 \rangle \in r$

- Note that since the behavior of Z and W are identical it follows that $Y \twoheadrightarrow Z$ if $Y \twoheadrightarrow W$

Example (Cont.)

- In our example:

$ID \twoheadrightarrow child_name$

$ID \twoheadrightarrow phone_number$

- The above formal definition is supposed to formalize the notion that given a particular value of Y (ID) it has associated with it a set of values of Z ($child_name$) and a set of values of W ($phone_number$), and these two sets are in some sense independent of each other.
- Note:
 - If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$
 - Indeed we have (in above notation) $Z_1 = Z_2$

Theory of MVDs

- From the definition of multivalued dependency, we can derive the following rule:

If $\alpha \rightarrow \beta$, then $\alpha \twoheadrightarrow \beta$

That is, every functional dependency is also a multivalued dependency

- The **closure D^+** of D is the set of all functional and multivalued dependencies logically implied by D .

Fourth Normal Form

- A relation schema R is in **4NF** with respect to a set D of functional and multivalued dependencies if for all multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following hold:
 - ◆ $\alpha \twoheadrightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$ or $\alpha \cup \beta = R$)
 - ◆ α is a superkey for schema R
- If a relation is in 4NF it is in BCNF

4NF Decomposition Algorithm

result := {*R*};

done := false;

compute D^+ ;

Let D_i denote the restriction of D^+ to R_i

while (**not** *done*)

if (there is a schema R_i in *result* that is not in 4NF) **then**

begin

 let $\alpha \twoheadrightarrow \beta$ be a **nontrivial multivalued dependency** that holds
 on R_i such that $\alpha \rightarrow R_i$ is not in D_i , and $\alpha \cap \beta = \emptyset$;

result := (*result* - R_i) $\cup$ (R_i - β) $\cup$ (α, β);

end

else *done* := true;

Note: each R_i is in 4NF, and decomposition is lossless-join

Example

$R = (A, B, C, G, H, I)$

$F = \{ A \twoheadrightarrow B$

$B \twoheadrightarrow HI$

$CG \twoheadrightarrow H \}$

- R is not in 4NF since $A \twoheadrightarrow B$ and A is not a superkey for R

- Decomposition

a) $R_1 = (A, B)$ (R_1 is in 4NF)

b) $R_2 = (A, C, G, H, I)$ (R_2 is not in 4NF, decompose into R_3 and R_4)

c) $R_3 = (C, G, H)$ (R_3 is in 4NF)

d) $R_4 = (A, C, G, I)$ (R_4 is not in 4NF, decompose into R_5 and R_6)

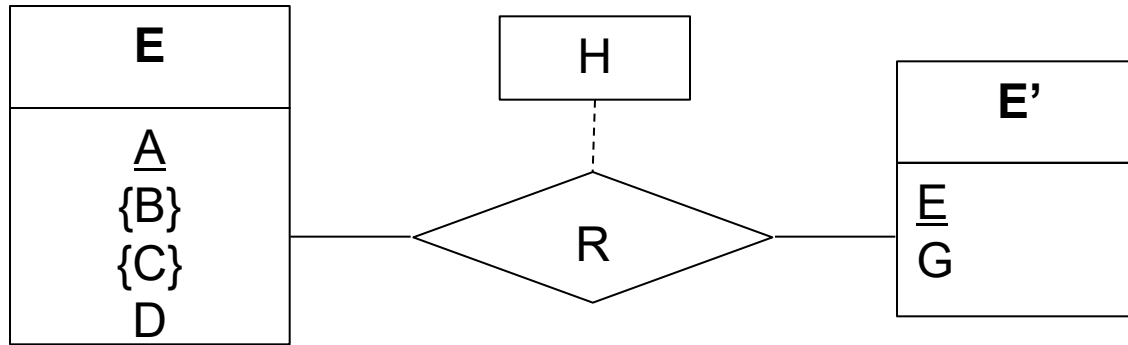
$A \twoheadrightarrow B$ and $B \twoheadrightarrow HI \Rightarrow A \twoheadrightarrow HI$, (MVD transitivity), and

and hence $A \twoheadrightarrow I$ (MVD restriction to R_4)

e) $R_5 = (A, I)$ (R_5 is in 4NF)

f) $R_6 = (A, C, G)$ (R_6 is in 4NF)

E-R Modeling and Normal Forms

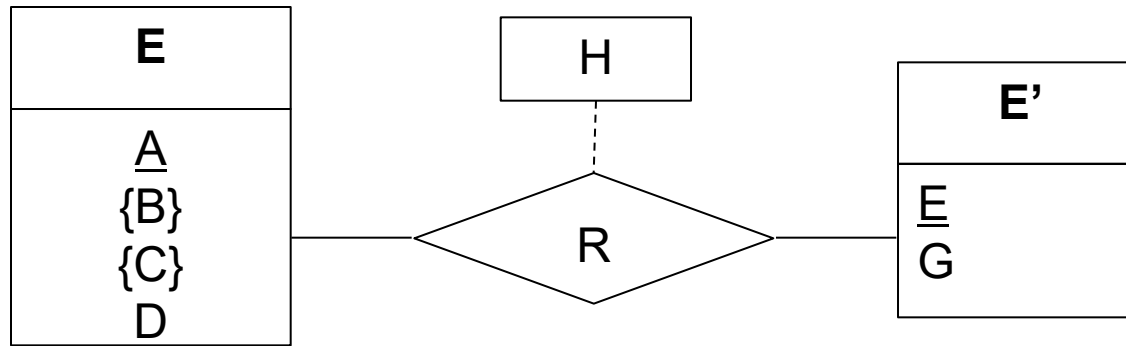


■ Above is an E-R diagram that depicts a many-to-many relationship R between two entity sets E and E'. The candidate key of E and E' are underlined in the diagram. B and C are multivalued attributes of Entity E. If the Entity E is transform to a relation schema E(A,B,C,D).

Please answer following questions:

1. Give the no-trivial functional dependency and multivalued dependency set D that holds on E.
2. List the candidate key(s) of E.
3. Prove that E is not in 4NF.
4. Explain the issues exist with E.
5. Please decompose E into 4NF relations.

E-R Modeling and Normal Forms



■ Please answer following questions:

1. Give the no-trivial functional dependency and multivalued dependency set D that holds on E.

$D = \{A \rightarrow D, A \twoheadrightarrow B, A \twoheadrightarrow C\}$

2. List the candidate key(s) of E : ABCD

3. Prove that E is not in 4NF.

$A \twoheadrightarrow B$ is no-trivial , and A is not a key.

4. Explain the issues exist with E.

Information repetition, Insertion anomalies, Update difficulty

5. Please decompose E into 4NF relations.

$R1 (A, B) , R2 (A, C), R3(A, D)$

Overall Database Design Process

- We have assumed schema R is given
 - ◆ R could have been generated when converting E-R diagram to a set of tables.
 - ◆ R could have been a single relation containing *all* attributes that are of interest (called **universal relation**). Normalization breaks R into smaller relations.
 - ◆ R could have been the result of some *ad hoc design of relations*, which we then test/convert to normal form.

ER Model and Normalization

- When an E-R diagram is carefully designed, identifying all entities correctly, the tables generated from the E-R diagram should not need further normalization.
- However, in a real (imperfect) design, there can be functional dependencies from non-key attributes of an entity to other attributes of the entity
 - Example: an *employee* entity with attributes *department_name* and *building*, and a functional dependency *department_name* → *building*
 - Good design would have made department an entity

Denormalization for Performance

- May want to use **non-normalized schema for performance**
- For example, displaying *prereqs* along with *course_id* and *title* requires join of *course* with *prereq*
- **Alternative 1:** Use denormalized relation containing attributes of *course* as well as *prereq* with all above attributes
 - faster lookup
 - extra space and extra execution time for updates
 - extra coding work for programmer and possibility of error in extra code
- **Alternative 2:** use a materialized view defined as
course ⋈ *prereq*
 - Benefits and drawbacks same as above, except no extra coding work for programmer and avoids possible errors

Other Design Issues

- Some aspects of database design are not caught by normalization
- Examples : 3 approaches to represent the same information:
 - ◆ 1. *earnings* (*company_id*, *year*, *amount*)
 - ◆ 2. *ernings_2012*(*company_id*, *earnings*)
earnings_2013(*company_id*, *earnings*)
earnings_2014(*company_id*, *earnings*)
 - ▶ Above are in BCNF, but make querying across years difficult and needs new table each year
 - ◆ 3. *company_year* (*company_id*, *earnings_2012*, *earnings_2013*, *earnings_2014*)
 - ▶ Also in BCNF, but also makes querying across years difficult and requires new attribute each year.
 - ▶ Is an example of a **crosstab**, where values for one attribute become column names
 - ▶ Used in spreadsheets, and in data analysis tools¹

End of Chapter 7